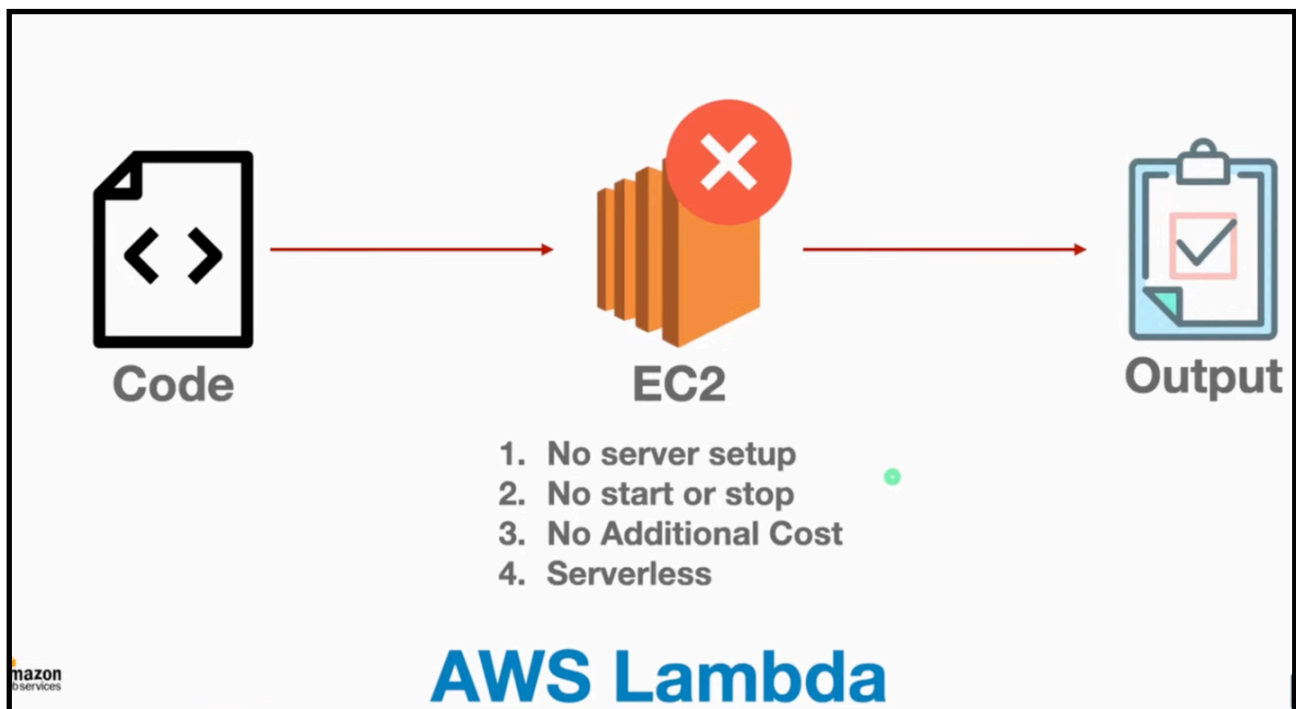


AWS Lambda

—-> What is AWS Lambda?

AWS Lambda is a **serverless compute service** that allows you to run code without provisioning or managing servers. You upload your code, and AWS automatically handles infrastructure, scaling, and availability.

You pay **only when the code is executed**.



—> Key Features of AWS Lambda

- No server management
- Automatic scaling
- High availability
- Pay-per-use pricing
- Supports multiple programming languages (Python, Node.js, Java, etc.)

Task 1: Create Your First Lambda Function

Goal

Understand **what Lambda is** and how it runs code.

—> Steps

Go to **AWS Console** → **Lambda**

Click **Create function**

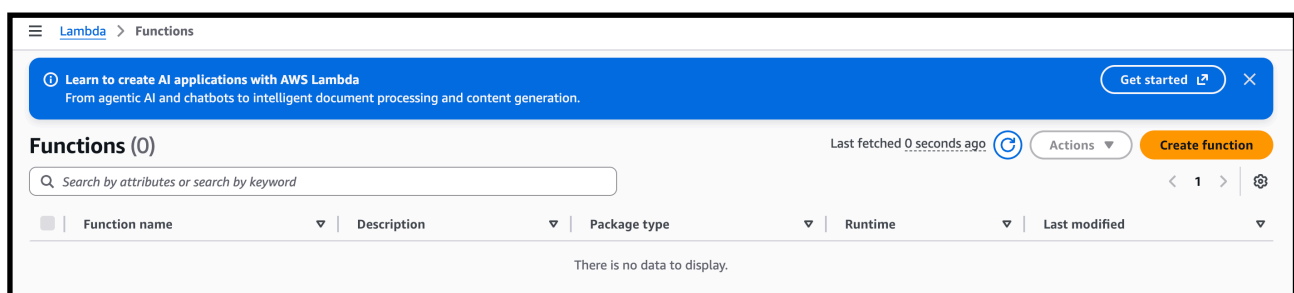
Select **Author from scratch**

Function name: **hello-lambda**

Runtime: **Python 3.x**

Execution role: **Create new role**

Click **Create function**



Function name: `hello-lambda`

Runtime: **Python 3.x**

Execution role: **Create new role**

Click **Create function**

The screenshot shows the 'Create function' page in the AWS Lambda console. The breadcrumb navigation at the top reads 'Lambda > Functions > Create function'. The main heading is 'Create function' with an 'Info' link. Below this, a prompt says 'Choose one of the following options to create your function.' There are three radio button options: 'Author from scratch' (selected), 'Use a blueprint', and 'Container image'. The 'Author from scratch' option has a subtext 'Start with a simple Hello World example.' Below these options is the 'Basic information' section. It contains three sub-sections: 'Function name' with a text input field containing 'hello-lambda' and a note about naming rules; 'Runtime' with a dropdown menu set to 'Python 3.14' and a refresh icon; and 'Architecture' with radio buttons for 'arm64' and 'x86_64' (selected). Below this is the 'Permissions' section, which includes a note about the default execution role and a 'Change default execution role' section. This section has a heading 'Execution role' and three radio button options: 'Create a new role with basic Lambda permissions' (selected), 'Use an existing role', and 'Create a new role from AWS policy templates'.

Lambda > Functions > Create function

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy.

Basic information

Function name
Enter a name that describes the purpose of your function.

hello-lambda

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.14

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.

☐ arm64

☒ x86_64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

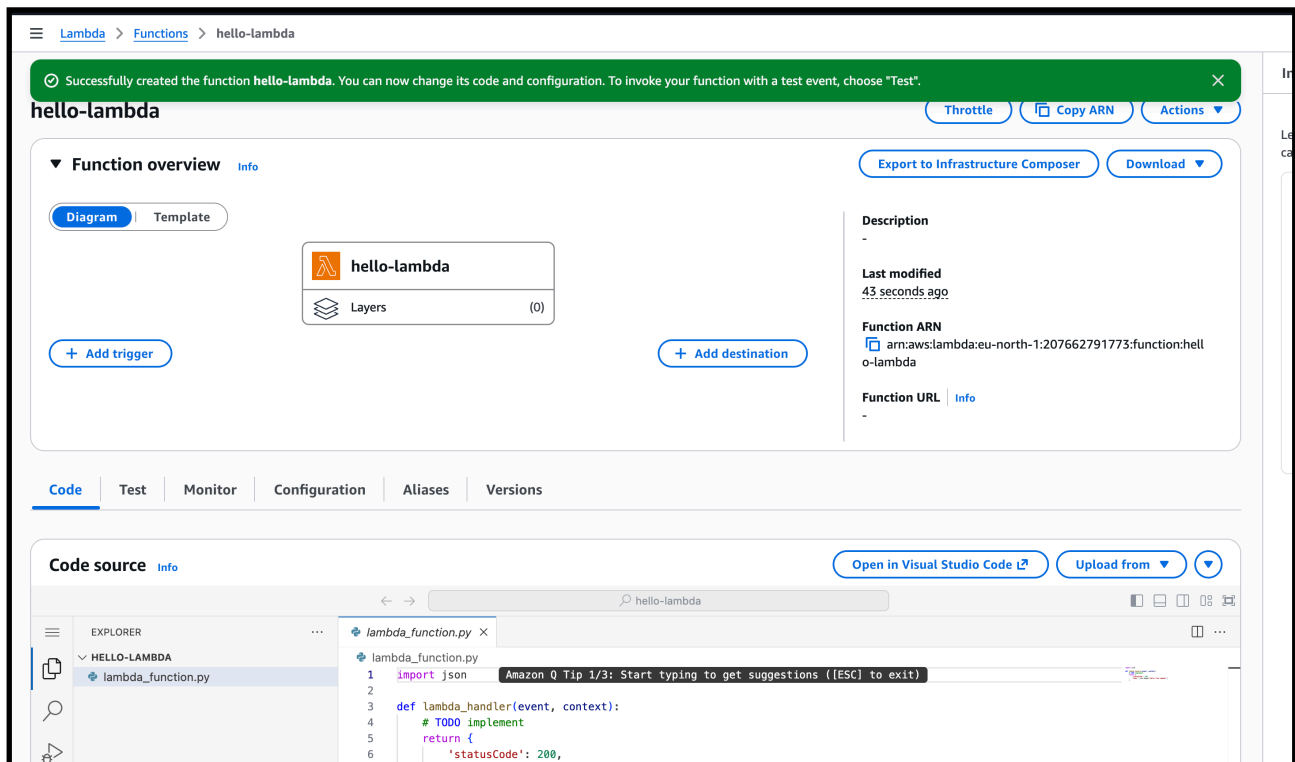
Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☒ Create a new role with basic Lambda permissions

☐ Use an existing role

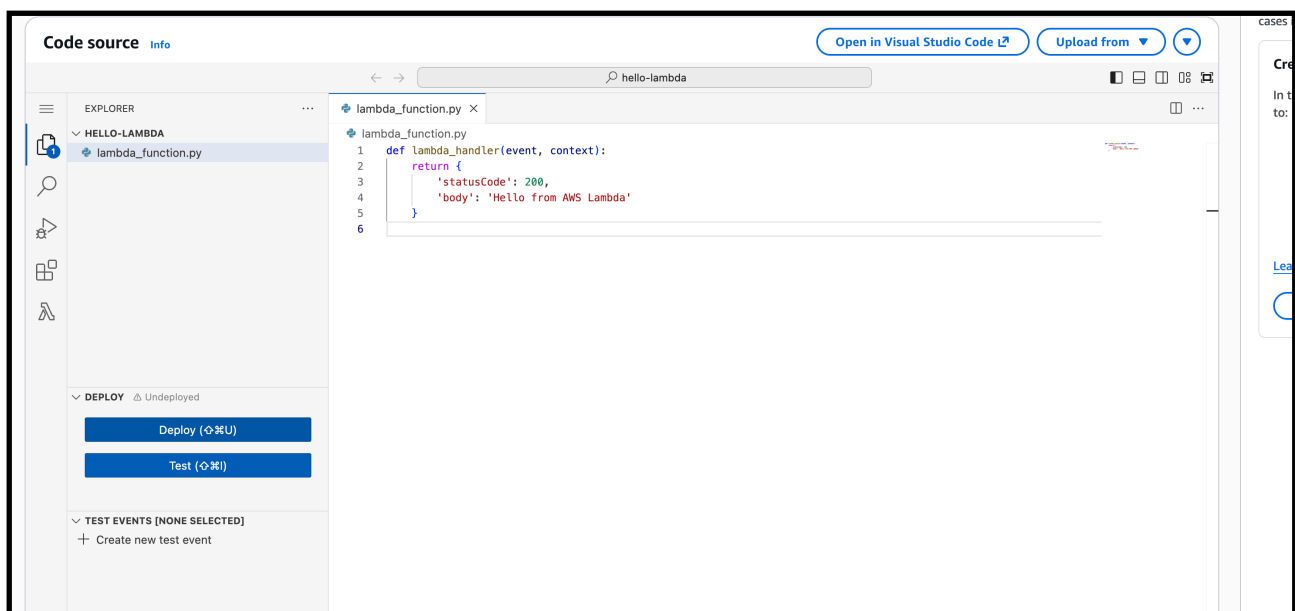
☐ Create a new role from AWS policy templates

—-> after click create our lambda function is created



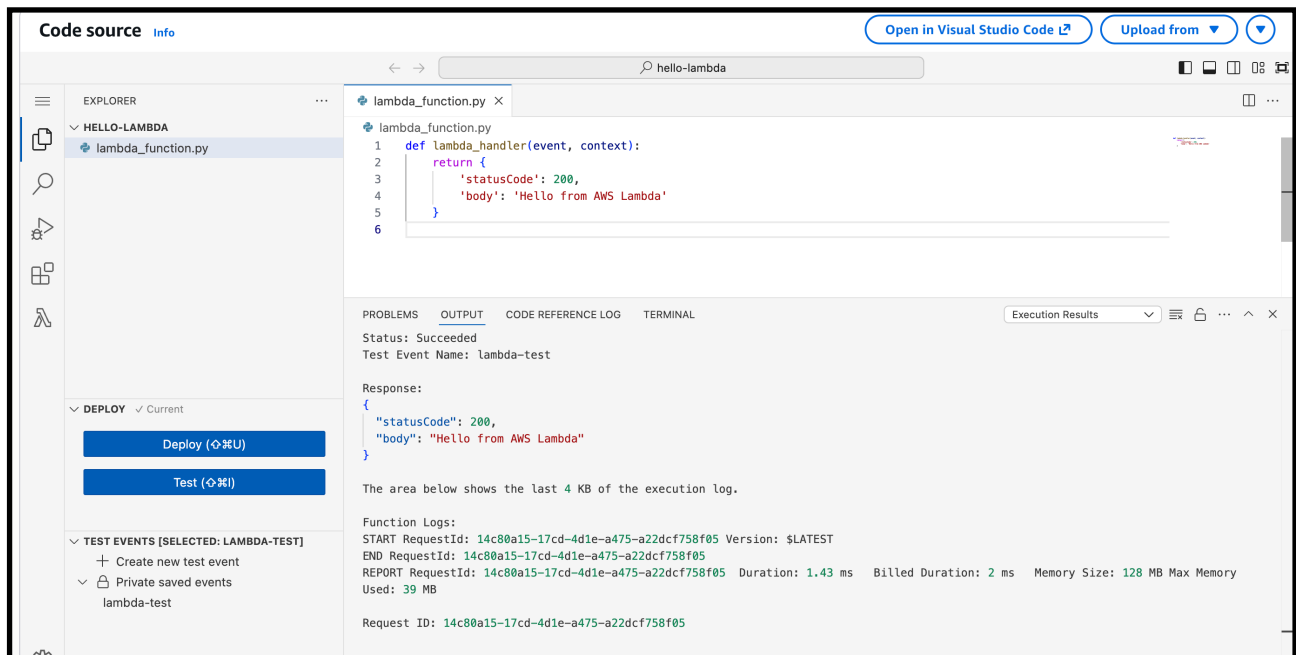
Code (replace existing code)

```
def lambda_handler(event, context):
    return {
        'statusCode': 200,
        'body': 'Hello from AWS Lambda'
    }
```



- Click **Test**

Output: "Hello from AWS Lambda"



—> Task 2 : Connect Lambda to API Gateway

Goal

Call Lambda using **browser URL**.

1)steps:

1. Go to **API Gateway**
2. Create **HTTP API**
3. Integration → **Lambda**
4. Select your function
5. Create route:
 - Method: **GET**
 - Path: **/hello**
6. Deploy
7. Copy **Invoke URL**

8. Open in browser

The screenshot shows the 'Configure API' step in the AWS API Gateway console. The left sidebar indicates the current step is 'Configure API'. The main content area is titled 'Configure API' and contains the following sections:

- API details**
 - API name:** A text input field containing 'lamda-api'.
 - IP address type:** Radio buttons for 'IPv4' (selected) and 'Dualstack'.
 - Integrations (1):** A dropdown menu showing 'Lambda'. Below it, fields for 'AWS Region' (eu-north-1), 'Lambda function' (arn:aws:lambda:eu-north-1:207662791773:function:hello-lambda), and 'Version' (2.0) are visible. A 'Remove' button is next to the integration.

At the bottom right, there are buttons for 'Cancel', 'Review and create', and 'Next'.

- Path: /hello
- Select the lambda which we created

The screenshot shows the 'Configure routes - optional' step in the AWS API Gateway console. The left sidebar indicates the current step is 'Configure routes'. The main content area is titled 'Configure routes - optional' and contains the following sections:

- Configure routes**
 - Method:** A dropdown menu showing 'GET'.
 - Resource path:** A text input field containing '/hello'.
 - Integration target:** A dropdown menu showing 'hello-lambda'. A 'Remove' button is next to it.

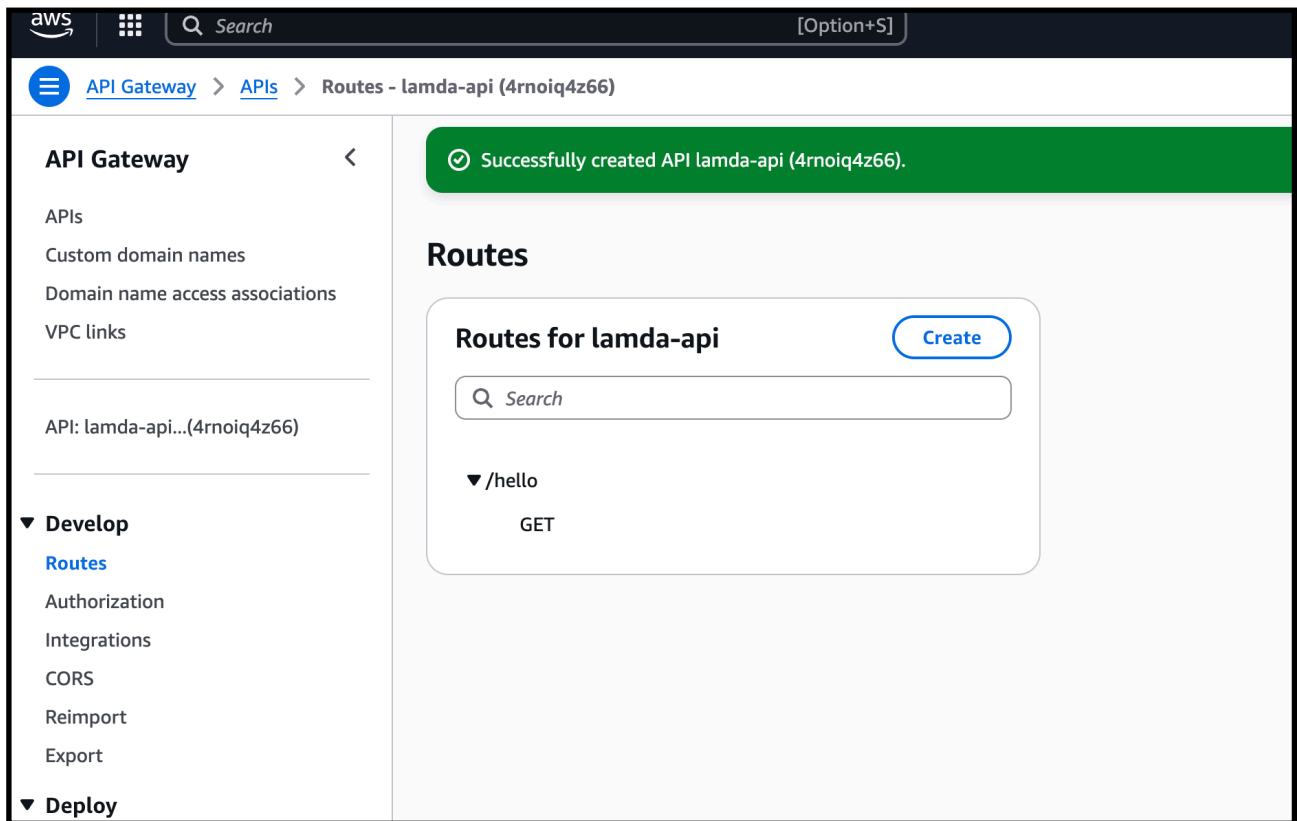
At the bottom right, there are buttons for 'Cancel', 'Review and create', 'Previous', and 'Next'.

—> review and create the api

The screenshot shows the 'Review and create' step in the AWS API Gateway console. The left sidebar indicates the current step is 'Review and create'. The main content area is titled 'Review and create' and contains the following sections:

- API name and integrations**
 - API name:** lamda-api
 - IP address type:** IPv4
 - Integrations:** hello-lambda (Lambda)
- Routes**
 - Routes:** GET /hello → hello-lambda (Lambda)
- Stages**
 - Stages:** \$default (Auto-deploy: enabled)

At the bottom right, there are buttons for 'Cancel', 'Previous', and 'Create'.



Successfully created API lamda-api

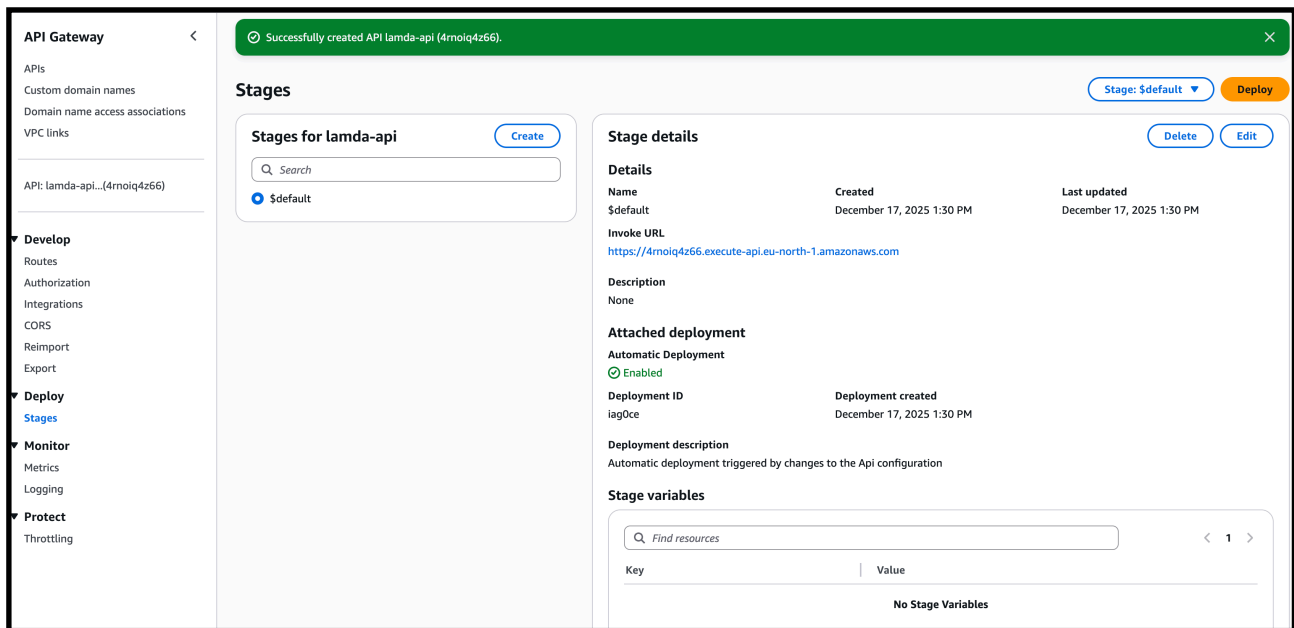
✓ Your HTTP API is live.

Go to API Gateway → APIs → lamda-api

2 Click Stages

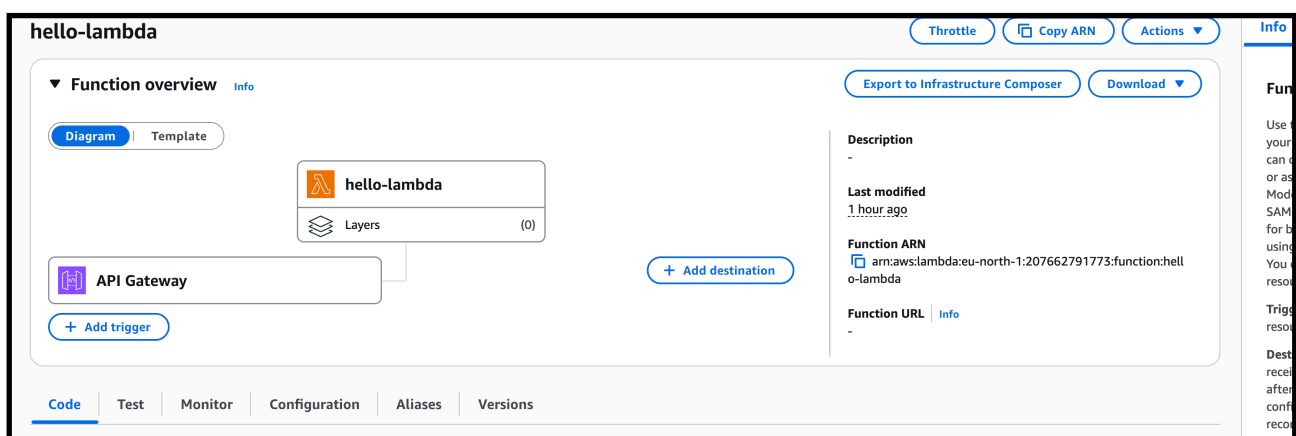
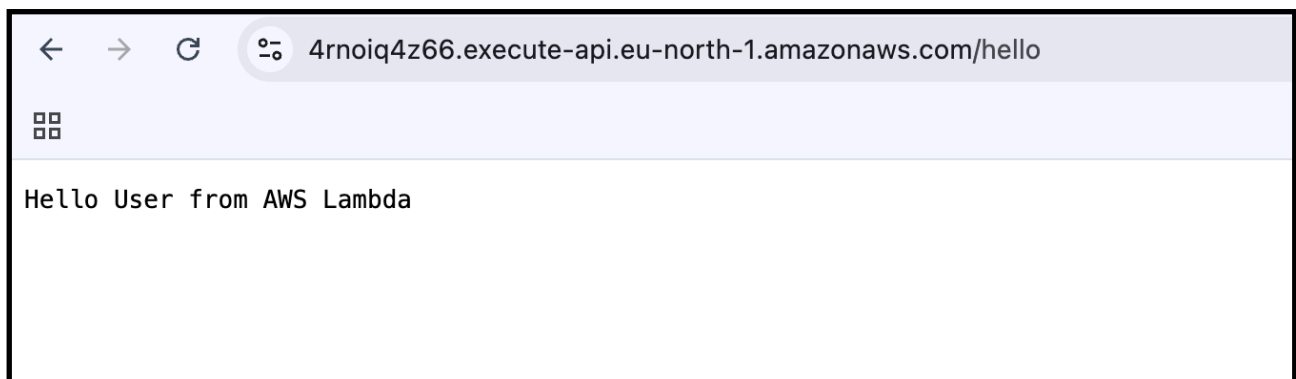
3 Click \$default

4 Copy Invoke URL



Open browser and add `/hello`

<https://4rnoi4z66.execute-api.eu-north-1.amazonaws.com/hello>



Task 3 : add trigger with cloudwatch

- You enabled **monitoring**
- You can debug Lambda failures
- You know where logs are stored

The screenshot shows the 'Add trigger' configuration page in the AWS Lambda console. The breadcrumb navigation at the top reads 'Lambda > Add triggers'. The main heading is 'Add trigger'. Below this is the 'Trigger configuration' section, which includes a dropdown menu for the trigger type, currently set to 'CloudWatch Logs'. Underneath, the 'Log group' field contains the ARN 'arn:aws:logs:eu-north-1:207662791773:log-group:my-api-lambda-logs:*'. The 'Filter name' field is labeled 'mylambdatrigger'. The 'Filter pattern - optional' field is empty. At the bottom right of the configuration area are 'Cancel' and 'Add' buttons.

Trigger configuration [Info](#)

CloudWatch Logs
aws asynchronous cw logging management-tools

Log group
Please select the CloudWatch Logs log group that serves as the event source. Log Events sent to the log group will trigger your Lambda function with the contents of the logs received.

Q X ©

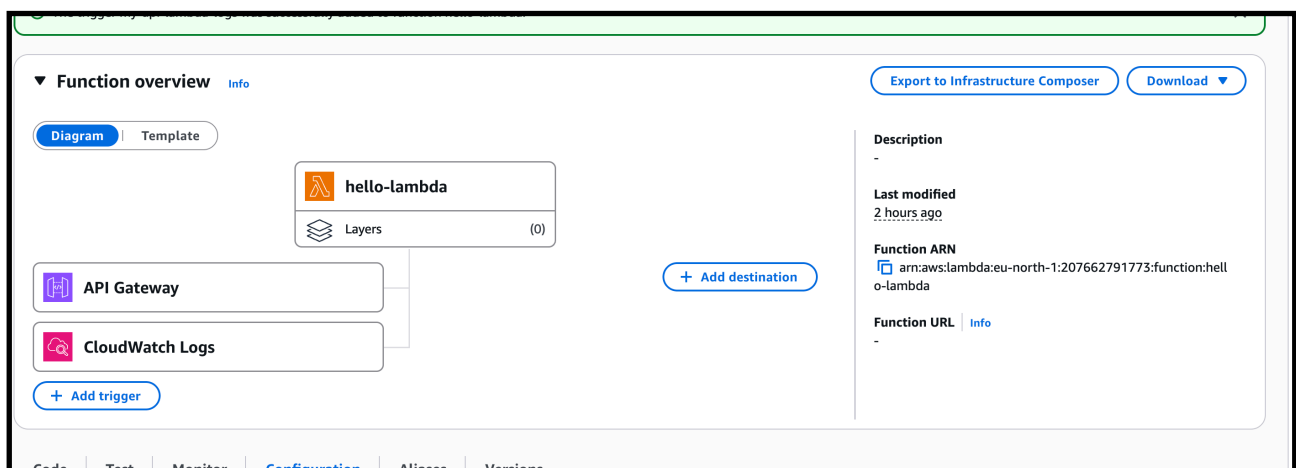
Select any log group except the log group for this function.

Filter name
Choose a name for your filter.

Filter pattern - optional
Enter an optional filter pattern.

Lambda will add the necessary permissions for Amazon CloudWatch Logs to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Cancel Add



Trigger 2: CloudWatch Logs

- Log group: `my-api-lambda-logs`
- Service principal: `logs.amazonaws.com`

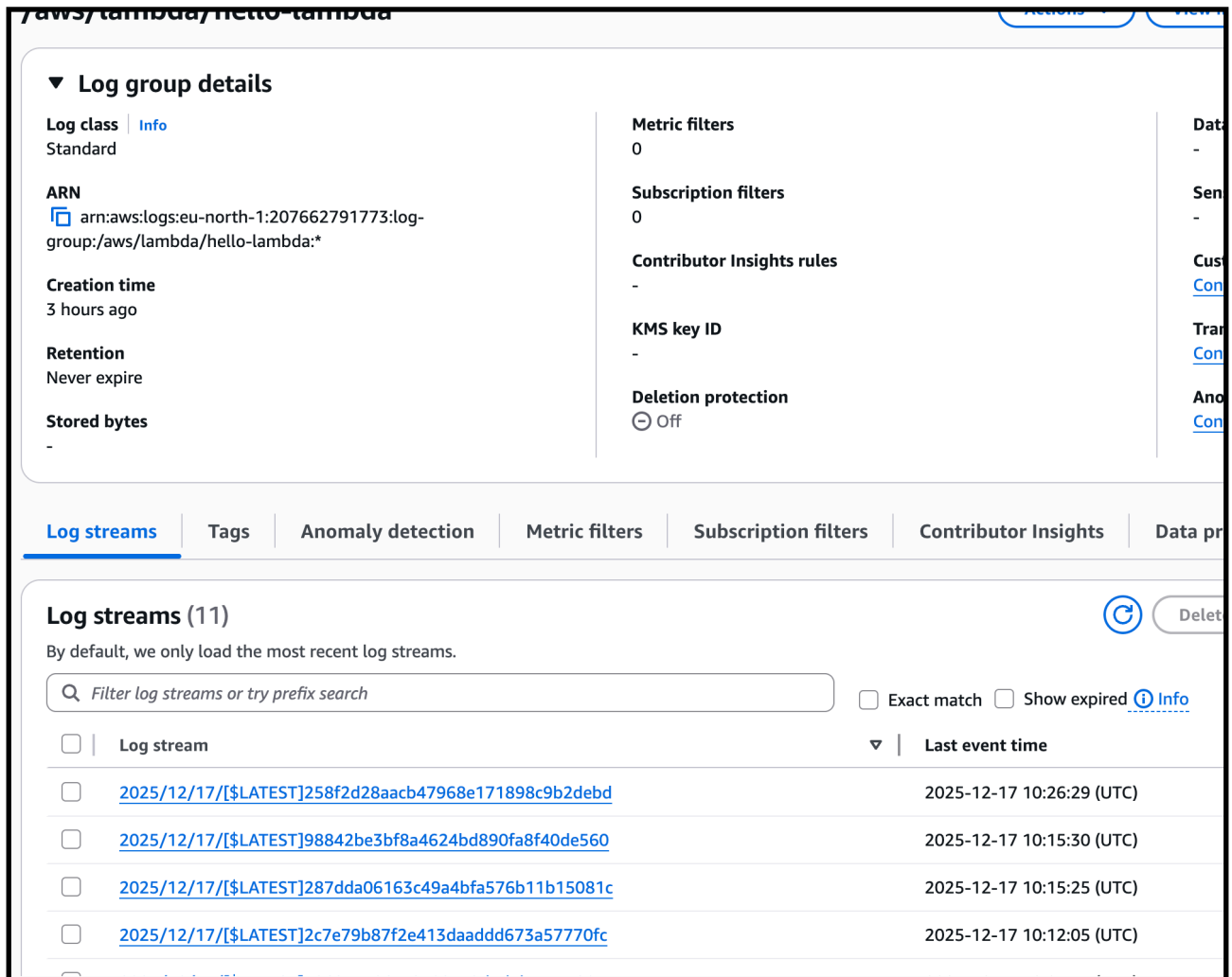
This means:

- You enabled **monitoring**
- You can debug Lambda failures
- You know where logs are stored

The screenshot shows the 'Triggers (2)' page in the AWS Lambda console. At the top, there are buttons for 'Fix errors', 'Edit', 'Delete', and 'Add trigger'. A search bar is also present. Below the search bar, there is a table of triggers. The first trigger is an 'API Gateway' trigger for the 'lamda-api' function. The second trigger is a 'CloudWatch Logs' trigger for the 'my-api-lambda-logs' log group. Both triggers have a 'Details' section expanded, showing various configuration options.

Trigger
API Gateway: lamda-api arn:aws:execute-api:eu-north-1:207662791773:4rnoiq4z66/*/*/hello API endpoint: https://4rnoiq4z66.execute-api.eu-north-1.amazonaws.com/hello ▼ Details API type: HTTP Authorization: NONE CORS: No Detailed metrics enabled: No isComplexStatement: No Method: ANY Resource path: /hello Service principal: apigateway.amazonaws.com Stage: \$default Statement ID: 0e3a7409-6cd7-57bf-87b0-3465428e21e8
CloudWatch Logs: my-api-lambda-logs arn:aws:logs:eu-north-1:207662791773:log-group:my-api-lambda-logs:* ▼ Details Filter name: mylambdatrigger Filter pattern: None isComplexStatement: No Service principal: logs.amazonaws.com Source account: 207662791773 Statement ID: lambda-1399df5e-4def-453a-a7a2-90323bcf26b8

—->now check the cloudwatch logs



Task 4: Lambda Error Handling & Logs

Objective

Demonstrate **error handling** and **error logs** in Lambda.

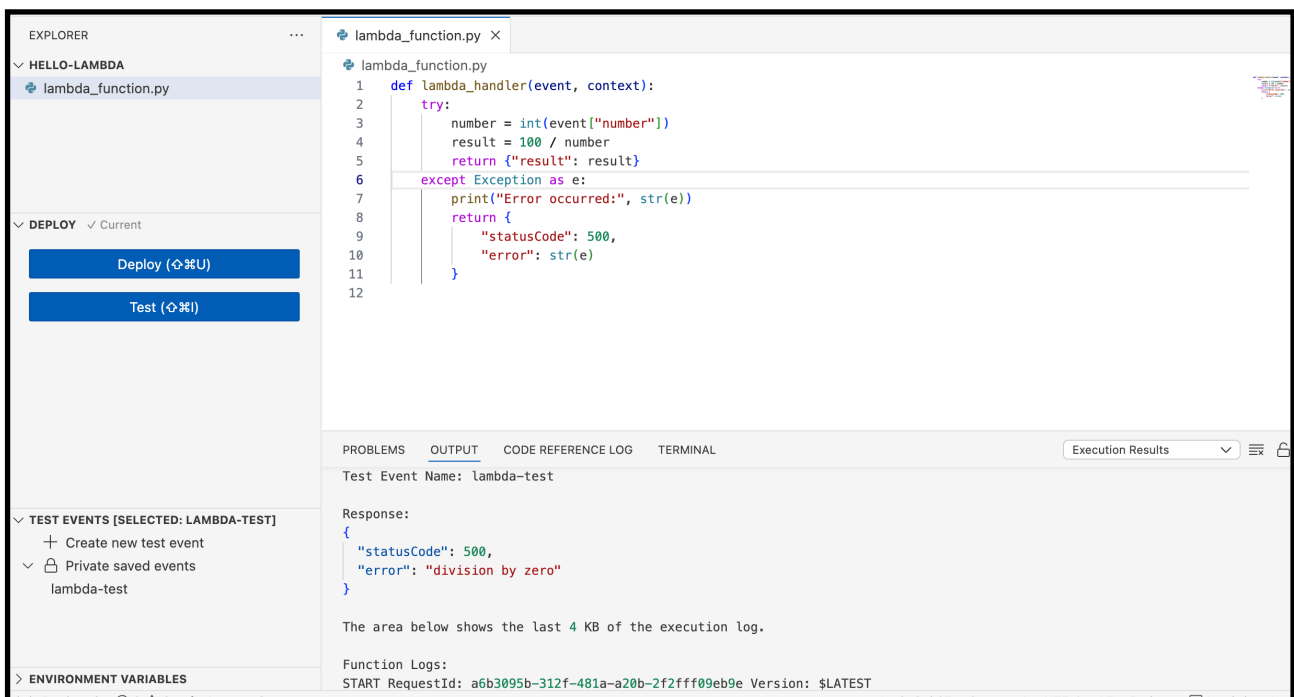
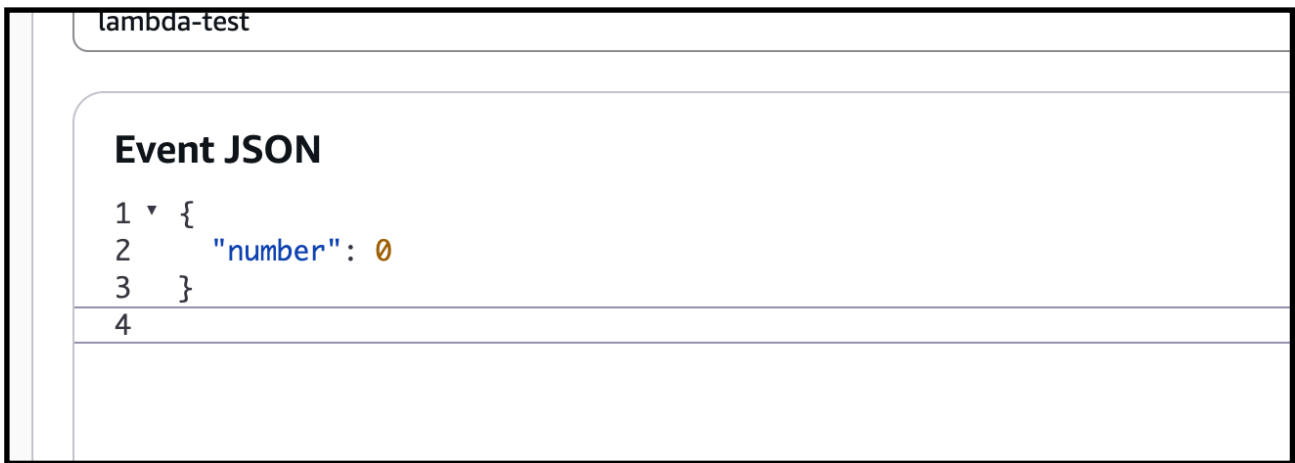
Lambda Code

```
def lambda_handler(event, context):
    try:
        number = int(event["number"])
```

```

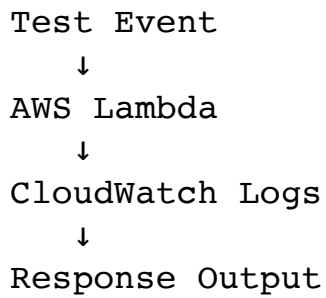
    result = 100 / number
    return {"result": result}
except Exception as e:
    print("Error occurred:", str(e))
    return {
        "statusCode": 500,
        "error": str(e)
    }

```



Check error message in **CloudWatch Logs**

Architecture Flow (Lambda Only)



Conclusion :

“I created an AWS Lambda function and exposed it using API Gateway. When a user hits the `/hello` endpoint, API Gateway triggers the Lambda function. Lambda executes the code and returns a response to the user. All execution logs are stored in CloudWatch Logs for monitoring.”